

MGR-SID: HIERARCHY-AWARE GRAPH-REGULARIZED SEMANTIC IDS FOR GENERATIVE RECOMMENDATION

Anonymous authors

Paper under double-blind review

ABSTRACT

Generative recommenders depend heavily on the quality of the item tokenizer that maps each item to a discrete semantic ID. Recent collaborative tokenizer designs usually inject one globally fused collaborative representation into the entire tokenizer, but our study of MiniOneRec suggests that this framing is too coarse for hierarchical semantic IDs. We find that the dominant tokenizer bottleneck is not full-code collision after generation, but local ambiguity inside semantically plausible prefixes, where the prefix is correct while the final leaf token remains hard to predict. Based on this observation, we propose **MGR-SID**, a hierarchy-aware graph-regularized tokenizer built on top of MiniOneRec’s semantic RQ-VAE backbone. The current **v1** instantiation uses a train-only graph bank with a purified coarse collaborative graph, a spectral mid-resolution graph, and a purified local transition graph, and applies different graph regularizers to different SID levels. We first show on Industrial and Office diagnostic probes that spectral middle-resolution graph views are much stronger than heuristic middle views for same-prefix disambiguation. We then align the tokenizer training regime with the upstream MiniOneRec recipe and evaluate training-time graph regularization under this fair setting. On Industrial, the hierarchy-aware variant reduces the final generated index collision from 0.00353 to 0.00326 relative to the semantic baseline, and decreases the fraction of test targets in multi-leaf same- l_2 buckets from 68.94% to 61.31%. Pushing the improved SID into downstream SFT yields a more nuanced result: the hierarchy-aware tokenizer improves short-range ranking ($\text{NDCG@3} : 0.0777 \rightarrow 0.0802$) and lowers same-prefix evaluate errors, but it does not yet surpass the semantic baseline on broader $@10+$ metrics. These results suggest that graph-structured collaboration is most useful when it enters SID learning as level-specific structural supervision rather than uniform feature fusion, while the tokenizer-to-SFT transfer remains only partially solved.

1 INTRODUCTION

Generative recommendation replaces ranking over a catalog with sequence generation over item codes. This reformulation has become increasingly attractive because it allows recommender systems to inherit the training and decoding machinery of language models. Recent systems such as TIGER, LCRec, and MiniOneRec demonstrate that a discrete item tokenizer can be the bridge between item semantics and autoregressive recommendation (Rajput et al., 2023; Zheng et al., 2023; Kong et al., 2025). Once the tokenizer is fixed, however, its structural biases are inherited by every downstream stage, including constrained decoding, supervised fine-tuning, and recommendation-oriented reinforcement learning.

This makes tokenizer quality a first-order problem rather than a preprocessing detail. In our current MiniOneRec reproduction line, we repeatedly observed a pattern that does not fit the usual “collision is the core problem” narrative. After the official `sid-generate` collision-repair stage, the final code collision rate is already at the sub-percent level, yet many prediction errors remain. A closer look shows that these errors often occur after the model has already entered the correct semantic region. In other words, the prefix is frequently correct, but the final leaf token is not. This points to a more specific bottleneck: *local ambiguity inside semantically plausible subtrees*.

At the same time, existing collaborative tokenizer methods suggest that collaborative signals should influence item tokenization, but they often do so through a single globally fused collaborative representation (Fang et al., 2026; Liang et al., 2026; Wang et al., 2026; Liu et al., 2025). That design is natural for flat item representations, but it is less natural for hierarchical semantic IDs. Different levels in a semantic ID serve different structural roles. Upper levels act as coarse routing decisions, while deeper levels distinguish among increasingly similar candidates. A single collaborative view injected uniformly across all levels may therefore be both under-specified and unstable.

This paper develops the current state of that idea into a concrete tokenizer-side method. We present **MGR-SID**, a hierarchy-aware graph-regularized semantic ID framework that uses graph structure as the carrier of collaborative information. Rather than replacing the semantic tokenizer, MGR-SID keeps MiniOneRec’s semantic RQ-VAE backbone and augments it with three train-only graph views: a purified coarse collaborative graph, a middle-resolution spectral graph, and a purified local transition graph. In the current **v1** implementation, we instantiate hierarchy-awareness with a simple but explicit level-wise rule: coarse regularization at level 1, spectral middle-view regularization at level 2, and local transition regularization at level 3.

Our evidence proceeds in four steps. First, we run diagnostic probes on Industrial and Office and show that middle-resolution graph views are indispensable: the best spectral middle view substantially outperforms heuristic middle views on same- l_2 ambiguity buckets. Second, we trace the provenance of the original MiniOneRec tokenizer and show that fair tokenizer comparison requires the upstream RQ-VAE training regime rather than the default local configuration. Third, under this aligned training setup, we show that hierarchy-aware graph regularization improves the *final generated SID* even though it does not dominate raw train-stage collision. Fourth, we carry the resulting tokenizer into Industrial SFT and find a partial downstream transfer: short-range ranking improves, and evaluate-time same-prefix errors decrease, but the broader end-to-end gain is not yet uniform.

The current draft is still centered on the tokenizer stage, but it now includes a first downstream SFT transfer study. We do not yet claim a new end-to-end best system or any RL improvement. Instead, the goal is to establish a precise tokenizer-side story and to test whether the resulting structural gains survive the first downstream stage.

Contributions. This draft makes three contributions.

1. We identify a tokenizer-side bottleneck for MiniOneRec: after fair reproduction of the upstream pipeline, the remaining difficulty lies in local leaf ambiguity rather than in unrepaired global collision.
2. We propose MGR-SID, a hierarchy-aware graph-regularized semantic ID framework that injects graph-structured collaboration into a semantic tokenizer through level-specific structural constraints instead of uniform feature fusion.
3. We provide preliminary but concrete evidence for this design. On Industrial, under upstream-aligned RQ-VAE training, hierarchy-aware regularization reduces final SID collision from 0.00353 to 0.00326 and improves local same- l_2 ambiguity metrics, while uniform graph regularization is clearly harmful. When the resulting SID is pushed into SFT, the hierarchy-aware variant improves short-range ranking and reduces same-prefix evaluate errors, even though it does not yet produce a broad @10+ improvement.

The remainder of the paper is organized as follows. Section 2 situates MGR-SID in prior work. Section 3 summarizes the tokenizer pipeline and the empirical bottlenecks that motivate our design. Section 4 presents the current MGR-SID v1 method. Section 5 reports probe studies, provenance alignment, tokenizer-stage results, and the first downstream SFT analysis. Section 6 concludes with limitations and the next steps toward stronger tokenizer-to-SFT transfer and RL evaluation.

2 RELATED WORK

Generative recommendation and semantic item IDs. Recent generative recommenders reformulate recommendation as constrained sequence generation over discrete item tokens. TIGER introduced generative retrieval for recommendation and showed that autoregressive decoding over item codes can replace traditional ranking (Rajput et al., 2023). LCRec aligned collaborative semantics with

language-model adaptation and demonstrated that tokenized item representations can be integrated into large language model pipelines (Zheng et al., 2023). MiniOneRec pushed this line further by releasing an end-to-end open-source framework that covers semantic ID construction, SFT, RL, and evaluation (Kong et al., 2025). These works establish the importance of tokenizer quality, but they mostly inherit a semantic-first tokenizer view. Our work stays within the MiniOneRec pipeline and asks a narrower question: how should collaborative structure enter a *hierarchical* semantic tokenizer without destabilizing it?

Collaborative-aware tokenization for generative recommenders. Several recent papers argue that text-only tokenization is insufficient. ReSID motivates recommendation-native tokenizer design from the perspective of prefix-conditional predictability and semantic quantization beyond plain language modeling (Liang et al., 2026). PRISM introduces purified collaborative representations and semantic anchoring to avoid noisy collaborative tokenization (Fang et al., 2026). PIT further highlights the volatility of collaborative signals and studies dynamic personalized tokenization (Wang et al., 2026). DiscRec separates semantic and collaborative factors before fusion (Liu et al., 2025). These methods all strengthen the case that collaboration matters, but they still leave open the question that matters most for hierarchical IDs: should the same collaborative signal be reused across every SID level, or should different levels preserve different structures? MGR-SID is built precisely around that missing allocation question.

Tokenization objectives beyond reconstruction. Another relevant thread asks tokenizers to preserve recommendation structure rather than optimizing reconstruction alone. ETEGRec argues that tokenization should be aligned with the recommender objective itself instead of being treated as an isolated preprocessor (Liu et al., 2024). CoST uses contrastive quantization to preserve semantic tokenization quality for generative recommendation (Zhu et al., 2024). These works are close in spirit to our method because they move the burden of recommendation structure into tokenizer learning. Our difference is that we use graph-structured collaborative signals as explicit structural targets and distribute them across SID levels, instead of introducing a single tokenization objective that treats all relations uniformly.

Graph signal processing and transition-aware recommendation. The graph view bank in MGR-SID is inspired by work showing that different graph scales capture different recommendation patterns. GSPRec decomposes graph signals into different spectral components for recommendation (Rabiah & McAuley, 2025), while FaGSP studies frequency-aware graph signal processing for collaborative filtering (Xia et al., 2024). Collaboration and Transition emphasizes that long-range collaborative structure and short-range transition structure are complementary rather than interchangeable (Zhu et al., 2023). These papers are not SID tokenizers, but they provide a strong clue for our design: the collaborative information most useful for a hierarchical tokenizer is likely multi-resolution, and middle-scale graph structure deserves explicit modeling rather than being approximated by a crude interpolation between global and local views.

Positioning of this work. MGR-SID combines ideas from all three threads above, but it does not aim to compete as a generic graph encoder or a full end-to-end personalized tokenizer. The current v1 is deliberately minimal. It inherits MiniOneRec’s semantic RQ-VAE backbone, borrows the denoising discipline of PRISM, uses the predictability-based motivation of ReSID, and instantiates graph-aware supervision with a coarse/mid/local graph bank whose middle view is selected by empirical probes inspired by GSPRec and FaGSP. In this sense, the paper’s main point is not “better graph features,” but *level-specific graph supervision for semantic ID learning*.

3 BACKGROUND AND PROBLEM FORMULATION

3.1 MINIONEREC TOKENIZER PIPELINE

Let each item i be associated with a semantic embedding $\mathbf{x}_i \in \mathbb{R}^d$. MiniOneRec tokenizes items by training an RQ-VAE over these item embeddings and then decoding each item into a fixed-length semantic ID $\text{SID}(i) = [z_i^{(1)}, z_i^{(2)}, z_i^{(3)}]$ (Kong et al., 2025; Lee et al., 2022). During `sid-train`, the model learns residual quantizers over the semantic embedding space. During `sid-generate`,

Table 1: Tokenizer provenance alignment on Industrial. The upstream-style RQ-VAE regime reproduces the low-collision official tokenizer quality, whereas the local default training configuration does not.

Setting	Epochs	Batch size	Best train collision	Final index collision
Local default rerun	10	256	0.9989	0.9940
Upstream-style rerun	10000	20480	0.0993	0.00434
Official committed index	–	–	–	0.00434

the model re-encodes colliding items with Sinkhorn-based assignment enabled so that the final exported `index.json` has much lower collision than the raw train-stage codes.

This separation is important. A tokenizer should not be judged only by train-stage collision. The actual pipeline uses `sid-generate` as part of the final tokenizer construction, so a fair comparison must examine the final generated SID rather than raw codes emitted directly by the training loop.

3.2 WHY LOCAL AMBIGUITY MATTERS MORE THAN UNREPAIRED GLOBAL COLLISION

Our diagnosis of the current semantic baseline consistently points to a local failure mode. We observed many cases where the model reaches the correct semantic prefix but fails at the final leaf. This means that the tokenizer still groups together items that are semantically close yet behaviorally distinct. We refer to this phenomenon as *local leaf ambiguity*.

To make this concrete, consider an item’s first two SID levels $(z_i^{(1)}, z_i^{(2)})$. The set of items sharing this two-level prefix defines a same- l_2 bucket. If many leaves remain under the same prefix, then the final decoder still faces a difficult local choice even when the upper routing is already correct. Our current analysis therefore treats same- l_2 crowding and the conditional entropy of the third-level code given the prefix as tokenizer health indicators in addition to collision.

3.3 FAIR TOKENIZER COMPARISON REQUIRES UPSTREAM-ALIGNED TRAINING

Before testing graph-aware tokenizers, we first had to resolve a provenance problem in the reproduced MiniOneRec pipeline. A direct rerun with the current local default configuration produced extremely high train-stage and generate-stage collision, which appeared inconsistent with the official low-collision Industrial index already stored in the repository. By comparing the local code path with the upstream MiniOneRec release and then rerunning with the upstream tokenizer recipe, we found that the discrepancy came primarily from hyperparameters rather than a broken implementation.

Table 1 is central to the rest of the paper. It shows that fair tokenizer comparison must use the upstream-style RQ-VAE training regime. All training-time comparisons for MGR-SID in this draft therefore adopt the same tokenizer backbone, optimizer, and training schedule as the upstream MiniOneRec recipe.

3.4 PROBLEM STATEMENT

These observations motivate a more specific tokenizer question than “should collaboration be fused into tokenization?” We instead ask:

How should graph-structured collaborative information be injected into different SID levels so that a semantic tokenizer becomes less locally ambiguous without collapsing its hierarchy?

The current MGR-SID v1 instantiation answers this question conservatively. It preserves the semantic RQ-VAE backbone and adds graph-based regularization only as a structural training signal. This allows us to test the value of hierarchy-aware graph supervision before modifying the downstream SFT or RL stages.

4 METHOD

4.1 OVERVIEW

MGR-SID keeps MiniOneRec’s semantic RQ-VAE tokenizer and augments it with a train-only graph bank. The core intuition is simple: different SID levels solve different structural discrimination problems, so they should preserve different collaborative graph structures. In the current **v1** implementation, we instantiate this idea with a fixed but non-uniform level-to-view assignment:

- level 1 is regularized by a coarse collaborative graph;
- level 2 is regularized by a middle-resolution spectral graph;
- level 3 is regularized by a local transition graph.

This is intentionally simpler than the full long-term vision of learned level-wise allocation. The purpose of v1 is to test whether hierarchy-aware structural supervision is already useful before adding another learned allocation module.

4.2 SEMANTIC BACKBONE

Given an item embedding $\mathbf{x}_i$, the encoder produces a latent representation that is quantized by three residual quantizers. Let $\mathbf{q}_i^{(l)}$ be the quantized vector produced by the l -th quantizer, and let

$$\mathbf{h}_i^{(l)} = \sum_{r=1}^l \mathbf{q}_i^{(r)} \quad (1)$$

be the cumulative quantized representation after level l . The decoder reconstructs $\mathbf{x}_i$ from $\mathbf{h}_i^{(3)}$. This is exactly the same semantic tokenizer backbone used by MiniOneRec, and we keep its reconstruction loss and residual quantization loss unchanged.

4.3 GRAPH BANK CONSTRUCTION

All graph views are constructed from training interactions only. For each interaction sample with history $H_t = [i_{t-k}, \dots, i_{t-1}]$ and target item y_t , we build three graph carriers.

Coarse collaborative graph. We build an undirected item-item graph from the unique items appearing in the recent history window and the target. For items u and v that co-occur inside the sequence, we add a symmetric edge weighted by inverse distance. This graph captures broad compatibility and stable collaborative organization. We then purify it by support pruning, popularity debiasing, and row normalization:

$$\mathbf{A}_{\text{coarse}} = \text{RowNorm}(\text{Debias}(\text{Prune}(\mathcal{G}_{\text{coarse}}))). \quad (2)$$

In the current implementation, pruning removes edges with weight below 2.0 and popularity debiasing uses exponent $\alpha = 0.5$.

Local transition graph. We build a directed graph from recent history items to the target item. Each edge receives inverse-rank weight, so more recent items contribute more. The purified local graph applies support pruning, target-popularity correction, and row normalization:

$$\mathbf{A}_{\text{local}} = \text{RowNorm}(\text{TargetDebias}(\text{Prune}(\mathcal{G}_{\text{local}}))). \quad (3)$$

This view captures short-range next-item preference but is naturally sparse and noisy.

Middle-resolution graph. The middle view is the key design choice. Early heuristic probes using diffusion residuals and simple band-pass proxies already suggested that mid-scale structure matters, but they were weaker than spectral graph constructions. The current v1 therefore selects a FaGSP-inspired spectral middle view as the default:

$$\mathbf{A}_{\text{mid}} = \text{BandReconstruct}(\text{SymNorm}(\mathbf{A}_{\text{coarse}})). \quad (4)$$

Operationally, we compute a band-limited spectral reconstruction of the purified coarse graph by retaining eigen-components in a middle spectral range. The implementation uses a rank of 48 and an eigen-ratio range of $[0.25, 0.65]$. We also explored a GSPRec-style temporal spectral view as an alternative, but use the FaGSP-style view as the main G_{mid} because it performed best in our diagnostic probes.

Semantic-anchor purification. Inspired by PRISM, we additionally tested a semantic-anchor purification stage in which a purified graph is intersected with a top- k semantic k NN graph extracted from the text embedding space. This module improves some graph views, but our probes indicate that the main gain comes from the quality of the middle-resolution graph itself rather than from anchor-only denoising. In the current training-time v1, the final chosen middle view is the spectral graph built from the purified coarse graph.

4.4 HIERARCHY-AWARE GRAPH REGULARIZATION

The semantic backbone already optimizes

$$\mathcal{L}_{\text{rec}} + \mathcal{L}_{\text{rq}}, \quad (5)$$

where $\mathcal{L}_{\text{rec}}$ is the reconstruction loss and $\mathcal{L}_{\text{rq}}$ is the residual quantization loss. MGR-SID v1 adds graph regularization on the cumulative representations $\mathbf{h}_i^{(l)}$.

For a mini-batch $\mathcal{B}$ and a row-normalized subgraph adjacency $\mathbf{A}_{\mathcal{B}}^{(l)}$ extracted from the selected graph view, we define

$$\mathcal{L}_{\text{graph}}^{(l)} = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \left\| \mathbf{h}_i^{(l)} - \sum_{j \in \mathcal{B}} \mathbf{A}_{\mathcal{B},ij}^{(l)} \mathbf{h}_j^{(l)} \right\|_2^2. \quad (6)$$

This loss favors level- l representations that are locally smooth on the corresponding graph view. Intuitively, items that are connected in the relevant graph should remain nearby at the level that is responsible for preserving that structure.

The hierarchy-aware v1 objective is then

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{rec}} + \mathcal{L}_{\text{rq}} + \lambda_1 \mathcal{L}_{\text{graph}}^{(1)} + \lambda_2 \mathcal{L}_{\text{graph}}^{(2)} + \lambda_3 \mathcal{L}_{\text{graph}}^{(3)}, \quad (7)$$

where

$$\mathbf{A}_{\mathcal{B}}^{(1)} \leftarrow \mathbf{A}_{\text{coarse}}, \quad \mathbf{A}_{\mathcal{B}}^{(2)} \leftarrow \mathbf{A}_{\text{mid}}, \quad \mathbf{A}_{\mathcal{B}}^{(3)} \leftarrow \mathbf{A}_{\text{local}}. \quad (8)$$

In the current implementation, $(\lambda_1, \lambda_2, \lambda_3) = (0.05, 0.15, 0.05)$.

4.5 CONTROLS

To isolate the effect of hierarchy-awareness, we compare MGR-SID v1 with two controls:

- **semantic baseline:** the upstream-aligned MiniOneRec RQ-VAE tokenizer with no graph regularization;
- **uniform regularization:** a control that averages the coarse, middle, and local graphs and applies one graph smoothness loss to the final cumulative representation only.

The uniform control keeps the same graph ingredients but removes the hierarchy-aware assignment. It is therefore the most direct test of whether the benefit comes from non-uniform level-specific supervision rather than from adding graph information in any form.

4.6 CURRENT SCOPE

MGR-SID v1 should be read as a tokenizer-stage instantiation, not the final method ceiling. It does *not* yet learn the graph allocation itself, and it does *not* modify downstream SFT or RL. The purpose of the current draft is narrower: to establish whether graph-structured collaborative information helps the semantic tokenizer *when injected as level-specific structural supervision*.

Table 2: Best middle-view probe results on same- l_2 buckets. Spectral middle views are much stronger than the first heuristic middle views, which motivated the use of a spectral G_{mid} in MGR-SID v1.

Dataset	Best heuristic middle view	Best spectral middle view	Spectral view name
Industrial	0.2747	0.5401	fagsp_mid_base
Office	0.2381	0.7619	fagsp_mid_base

5 EXPERIMENTS

5.1 EXPERIMENTAL SETUP

Datasets. We use the Industrial and Office Amazon benchmarks already prepared in the MiniOneRec reproduction pipeline. Industrial contains 36,259 training interactions, 4,533 test interactions, and 3,686 items. Office contains 38,924 training interactions, 4,866 test interactions, and 3,459 items. Both use the repository’s existing time-based splits.

Tokenizer backbone. Unless otherwise noted, all training-time tokenizer comparisons use the upstream-aligned MiniOneRec RQ-VAE regime established in Table 1: three quantizers with codebook size 256, embedding dimension 32, AdamW, learning rate 10^{-3} , 10,000 epochs, and batch size 20,480.

Evaluation metrics. We report three classes of metrics.

- **Probe metrics:** target-better rate on ambiguity buckets, especially same- l_2 .
- **Tokenizer metrics:** best train-stage collision, final `sid-generate` collision, and maximum conflict size.
- **Local ambiguity metrics:** mean target l_2 leaf count, the fraction of targets in multi-leaf same- l_2 buckets, the fraction of targets in deep crowded same- l_2 buckets, and conditional entropy of level 3 under a fixed level-1/2 prefix.

5.2 DESIGN-SPACE PROBES: SPECTRAL MIDDLE VIEWS MATTER

We began with a non-invasive probe protocol that does not modify the tokenizer. Given an existing top-1 SID error, we ask whether a graph view scores the ground-truth target item above the currently favored wrong candidate. This lets us study which graph view is most informative before injecting it into training.

Table 2 provides the first direct justification for the graph bank design. The diagnostic result is not just that graph signals help. Rather, the result is that *middle-resolution graph structure matters enough to deserve explicit design*. The same- l_2 gain from the best spectral middle view is nearly doubled on Industrial and more than tripled on Office relative to the first heuristic middle view. This is what led us to instantiate G_{mid} with a spectral graph rather than a crude diffusion proxy.

5.3 TRAINING-TIME COMPARISON UNDER UPSTREAM-ALIGNED TOKENIZER TRAINING

We next test whether the graph signal survives once it is injected into tokenizer learning. All variants here use the fair upstream-aligned training setup from Table 1. We compare the semantic baseline, the uniform regularization control, and the hierarchy-aware MGR-SID v1 regularizer.

Table 3 reveals two important facts. First, uniform graph regularization is clearly harmful. This matters because it shows that adding graph structure per se is not enough. Second, train-stage collision and final tokenizer quality are not aligned. The semantic baseline still has the best raw train-stage collision, but after the official `sid-generate` stage, the hierarchy-aware tokenizer becomes the best final index. This is exactly why our evaluation protocol treats the final exported SID as the primary tokenizer artifact.

Table 3: Industrial tokenizer results under the upstream-aligned MiniOneRec training regime. Train-stage collision alone is misleading: the hierarchy-aware variant becomes the best tokenizer after sid-generate.

Method	Best train collision	Final index collision	Max conflict
Semantic baseline	0.1066	0.00353	2
Uniform graph regularization	0.2458	0.00597	6
Hierarchy-aware MGR-SID v1	0.1319	0.00326	2

Table 4: Local ambiguity comparison on Industrial final indices. The hierarchy-aware tokenizer reduces both catalog-level and test-weighted same- l_2 crowding.

Metric	Baseline	Hierarchy	Delta
Mean target l_2 leaf count	4.7999	4.4498	-0.3501
Fraction targets in multi-leaf l_2	0.6894	0.6131	-0.0763
Fraction targets in l_2 with ≥ 4 leaves	0.3283	0.2828	-0.0454
Mean target level-3 entropy under l_2	1.4533	1.2935	-0.1598

5.4 DOES THE FINAL TOKENIZER ACTUALLY REDUCE LOCAL AMBIGUITY?

The main question is whether the better final collision reflects the failure mode we care about. To answer this, we compare the final baseline index and the final hierarchy-aware index at the level of same- l_2 crowding.

Table 4 shows that the improvement is structurally meaningful. The hierarchy-aware tokenizer does not merely shuffle code assignments while preserving the same ambiguity. It lowers the average number of competing leaves under the target prefix, reduces the fraction of targets that fall into multi-leaf same- l_2 buckets, and lowers the conditional entropy of the third-level code. In the full movement analysis, 17.32% of test targets move from a multi-leaf same- l_2 bucket into a singleton bucket, while only 9.68% move in the opposite direction. This strongly supports our original diagnosis that the remaining tokenizer bottleneck is local leaf ambiguity.

5.5 FIRST DOWNSTREAM TEST: V1 TRANSFERS ONLY PARTIALLY

We first pushed the final Industrial tokenizers from Tables 3 and 4 into downstream SFT. Here, the internal comparison uses the upstream-aligned semantic run and the hierarchy-aware v1 run under the same controlled pipeline. We emphasize that this *internal control* is useful for diagnosis, but it is not our final baseline anchor.

Table 5 was the first indication that tokenizer gains do transfer downstream, but only partially. Relative to the internal semantic control, v1 improves NDCG@3 and HR@3, and slightly improves @5, but remains weaker at @10+. This pattern motivated the next step: instead of further changing the graph carrier, we redesigned the training interface itself.

5.6 AMBIGUITY-AWARE TOKENIZER V2

The top- k and error-distribution analysis of v1 suggested that graph regularization was helpful on crowded local ambiguity cases but over-corrected already-stable semantic regions. We therefore introduced an ambiguity-aware v2 tokenizer with two changes:

- an *offline ambiguity prior* built from semantic density, semantic-collaborative disagreement, and graph competition strength;
- a *semantic-structure retention* regularizer that preserves the original semantic neighborhood in low-ambiguity regions.

The first v2 tokenizer run used only the offline combined ambiguity prior, because the current online uncertainty proxy was not reliable enough in preliminary sanity checks.

Table 5: Industrial SFT results for the internal control comparison. MGR-SID v1 improves short-range ranking but does not yet form a broad @10+ gain.

Run	N@1	N@3	N@10	H@1	H@3	H@10
Internal semantic control	0.0631	0.0777	0.0943	0.0631	0.0882	0.1343
Internal hierarchy-aware v1	0.0627	0.0802	0.0936	0.0627	0.0929	0.1304

Table 6: Tokenizer-side comparison after introducing ambiguity-aware supervision. The v2 tokenizer does not further reduce final collision, but produces the cleanest local structure.

Method	Final collision	Mean target l_2 leaves	Multi-leaf same- l_2	Mean level-3 entropy
Internal semantic control	0.00353	4.7999	0.6894	1.4533
Hierarchy-aware v1	0.00326	4.4498	0.6131	1.2935
Ambiguity-aware v2	0.00353	4.3422	0.4873	1.1001

Table 6 is important for two reasons. First, it shows that collision is not the whole tokenizer story: v2 only ties the internal semantic control on final collision, yet it yields the cleanest local hierarchy among all tested tokenizers. Second, the ambiguity-aware design does exactly what it was intended to do: it reduces the fraction of targets that remain trapped in multi-leaf same- l_2 buckets without simply increasing graph pressure everywhere.

5.7 DOWNSTREAM SFT UNDER THE RECIPE-ALIGNED ORIGINAL MINIONEREC SETTING

We first evaluate whether this cleaner v2 tokenizer helps in a fair downstream comparison against the *original* MiniOneRec under the same task recipe. The first v2 downstream run uses:

- `title_history2sid = on`
- `SID-description alignment = off`

Hence the recipe-aligned original MiniOneRec comparison is the historical run with the same task setting. For context, we also report the strongest original MiniOneRec SFT and RL results in the repository.

Table 7 establishes the current position of our method much more clearly than the earlier internal control. Relative to the *recipe-aligned original MiniOneRec*, v2 improves all reported NDCG cutoffs:

$$\Delta\text{NDCG}@1 = +0.0079, \Delta\text{NDCG}@3 = +0.0041, \Delta\text{NDCG}@5 = +0.0036, \Delta\text{NDCG}@10 = +0.0021.$$

It also improves HR@1, HR@3, and HR@5, while trailing slightly on HR@10.

This means the ambiguity-aware tokenizer is no longer just “working on an internal control.” It already improves downstream recommendation quality against the original MiniOneRec under the same task recipe. At this stage, however, it still does *not* surpass the strongest original MiniOneRec recipe, which disables `title_history2sid` and enables description alignment with probability 0.5.

5.8 RECIPE ISOLATION: THE MAIN MISMATCH IS `TITLE_HISTORY2SID_OFF`

To understand why the strongest original recipe did not transfer cleanly to v2, we completed the missing cells of the v2 recipe matrix on Industrial:

- `title_history2sid_on + desc_align_off`
- `title_history2sid_on + desc_align_p05`
- `title_history2sid_off + desc_align_off`
- `title_history2sid_off + desc_align_p05`

Table 7: Industrial downstream comparison for v2. V2 already surpasses the recipe-aligned original MiniOneRec on all reported NDCG cutoffs and on HR@1/3/5, but does not yet surpass the strongest original MiniOneRec recipe.

Run	N@1	N@3	N@10	H@1	H@3	H@10
Recipe-aligned original MiniOneRec	0.0624	0.0798	0.0987	0.0624	0.0929	0.1471
Strongest original MiniOneRec SFT	0.0671	0.0850	0.1037	0.0671	0.0984	0.1509
Strongest original MiniOneRec RL	0.0732	0.0890	0.1073	0.0732	0.1004	0.1513
Ambiguity-aware v2	0.0704	0.0839	0.1008	0.0704	0.0942	0.1425

Table 8: Recipe isolation for the v2 tokenizer on Industrial. The best downstream recipe for v2 is title_history2sid_on + desc_align_p05, and the dominant negative factor is turning title_history2sid off.

Run	N@1	N@3	N@10	H@1	H@3	H@10
v2_on_off	0.0704	0.0839	0.1008	0.0704	0.0942	0.1425
v2_on_p05	0.0706	0.0845	0.1027	0.0706	0.0951	0.1463
v2_off_off	0.0589	0.0741	0.0913	0.0589	0.0854	0.1339
v2_off_p05	0.0591	0.0737	0.0899	0.0591	0.0843	0.1308

Table 8 resolves the strongest-recipe mismatch. When title_history2sid stays on, adding description alignment with probability 0.5 is beneficial:

$$\Delta\text{NDCG@10} = +0.0019, \quad \Delta\text{HR@10} = +0.0038.$$

By contrast, turning title_history2sid off causes a large drop regardless of whether description alignment is enabled. Under desc_align_off, the drop is

$$\Delta\text{NDCG@10} = -0.0096, \quad \Delta\text{HR@10} = -0.0086,$$

and under desc_align_p05 it becomes even larger:

$$\Delta\text{NDCG@10} = -0.0128, \quad \Delta\text{HR@10} = -0.0154.$$

This means the main negative factor is not description alignment itself. The real source of mismatch is title_history2sid_off. For the current v2 tokenizer, the best downstream setting is therefore title_history2sid_on + desc_align_p05.

We also compared this best v2 setting against the strongest original MiniOneRec SFT. The remaining gap is now very small:

$$\Delta\text{NDCG@10} = -0.0010, \quad \Delta\text{HR@10} = -0.0046.$$

Further top- k analysis shows that v2 already wins at rank 1 and produces much cleaner same-prefix error behavior, while the strongest original SFT still has an advantage in mid-beam neighborhood retention at top3/top5/top10. In other words, the remaining issue is no longer a tokenizer-collapse problem; it is a downstream interaction gap.

5.9 CURRENT INTERPRETATION

Taken together, the experiments support a more precise claim than our earlier v1-only narrative:

1. fair tokenizer comparison requires the upstream MiniOneRec training regime;
2. graph-structured collaborative information has real signal for local same- l_2 disambiguation, and this signal is strongest when encoded by a spectral middle-resolution graph;
3. uniform hierarchy-aware graph injection is insufficient because it over-corrects already-stable semantic regions;
4. ambiguity-aware graph supervision plus semantic-structure retention yields a tokenizer with a cleaner local hierarchy than both the internal semantic control and v1;

5. this cleaner tokenizer already improves downstream recommendation against the recipe-aligned original MiniOneRec;
6. after recipe isolation, the best downstream setting for the current v2 tokenizer is `title_history2sid_on + desc_align_p05`;
7. the strongest original MiniOneRec recipe remains slightly better overall, but the remaining gap is now small and is best described as a mid-beam neighborhood-retention gap rather than a tokenizer-side failure.

At the current stage, the main conclusion is therefore not “we already beat the whole system.” The stronger conclusion is:

the tokenizer-side method is now supported by structural evidence, recipe-aligned downstream evidence, and recipe-isolation evidence. The current best v2 setting is already very close to the strongest original MiniOneRec SFT, and the remaining gap is primarily a downstream retention issue rather than a failure of the ambiguity-aware tokenizer itself.

6 CONCLUSION AND LIMITATIONS

This draft studies the tokenizer stage of MiniOneRec and argues that graph-structured collaboration should enter semantic ID learning as level-specific structural supervision rather than uniform feature fusion. The resulting method, MGR-SID v1, keeps the semantic RQ-VAE backbone intact, builds a train-only graph bank with coarse, middle, and local views, and applies hierarchy-aware graph regularization to different SID levels. Under a fair upstream-aligned tokenizer regime, this design improves the final generated Industrial SID over the semantic baseline and reduces same- l_2 local ambiguity. When the improved SID is pushed into Industrial SFT, the gain transfers only partially: short-range ranking and crowded local cases improve, but broad @10+ recommendation quality does not yet improve consistently.

The current evidence is still limited in three ways. First, the full training-time result is currently based on Industrial only. Second, the current v1 uses a fixed level-to-view assignment rather than a learned allocation module. Third, although we now have a first SFT transfer study, the tokenizer gain has not yet turned into a new end-to-end best system, and RL remains untouched. These limits are intentional: the goal of the present draft is to establish a clean tokenizer-side story and then expose the precise point where tokenizer gains are only partially preserved downstream.

The next step is therefore more precise than before. Rather than immediately expanding to a larger method, we should improve the tokenizer-to-SFT transfer itself: preserve the hierarchy-aware reduction of local ambiguity while retaining the baseline’s stronger coarse-prefix stability. Once that interface is better aligned, the same tokenizer can be carried into repeated-seed SFT, Office evaluation, and then RL.

Reproducibility note. All experiments in this draft are built as incremental modules on top of the existing OneRec repository. The default baseline path remains unchanged, and the graph-aware tokenizer branch uses isolated configs, scripts, logs, and outputs.

REFERENCES

- Dengzhao Fang, Jingtong Gao, Yu Li, Xiangyu Zhao, and Yi Chang. Prism: Purified representation and integrated semantic modeling for generative sequential recommendation, 2026. URL <https://arxiv.org/abs/2601.16556>.
- Xiaoyu Kong, Leheng Sheng, Junfei Tan, Yuxin Chen, Jiancan Wu, An Zhang, Xiang Wang, and Xiangnan He. Minionerec: An open-source framework for scaling generative recommendation, 2025. URL <https://arxiv.org/abs/2510.24431>.
- Doyup Lee, Chiheon Kim, Saehoon Kim, Minsu Cho, and Wook-Shin Han. Autoregressive image generation using residual quantization. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 11513–11522. IEEE, 2022. doi: 10.1109/CVPR52688.2022.01123. URL <https://doi.org/10.1109/CVPR52688.2022.01123>.

-
- Yu Liang, Zhongjin Zhang, Yuxuan Zhu, Kerui Zhang, Zhiluo Han Guo, Wenhao Zhou, Zongqi Yang, Kangle Wu, Yabo Ni, Anxiang Zeng, Cong Fu, Jianxin Wang, and Jiazhi Xia. Rethinking generative recommender tokenizer: Recsys-native encoding and semantic quantization beyond lms, 2026. URL <https://arxiv.org/abs/2602.02338>.
- Chang Liu, Yimeng Bai, Xiaoyan Zhao, Yang Zhang, Fuli Feng, and Wenge Rong. Disrec: Disentangled semantic-collaborative modeling for generative recommendation, 2025. URL <https://arxiv.org/abs/2506.15576>.
- Enze Liu, Bowen Zheng, Cheng Ling, Lantao Hu, Han Li, and Wayne Xin Zhao. Generative recommender with end-to-end learnable item tokenization, 2024. URL <https://arxiv.org/abs/2409.05546>.
- Ahmad Bin Rabiah and Julian McAuley. Gsprec: Temporal-aware graph spectral filtering for recommendation, 2025. URL <https://arxiv.org/abs/2505.11552>.
- Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan Hulikal Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Q. Tran, Jonah Samost, Maciej Kula, Ed H. Chi, and Mahesh Sathiamoorthy. Recommender systems with generative retrieval. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/20dcab0f14046a5c6b02b61da9f13229-Abstract-Conference.html.
- Huanjie Wang, Xinchun Luo, Honghui Bao, Zixing Zhang, Lejian Ren, Yunfan Wu, Hongwei Zhang, Liwei Guan, and Guang Chen. Pit: A dynamic personalized item tokenizer for end-to-end generative recommendation, 2026. URL <https://arxiv.org/abs/2602.08530>.
- Jiafeng Xia, Dongsheng Li, Hansu Gu, Tun Lu, Peng Zhang, Li Shang, and Ning Gu. Frequency-aware graph signal processing for collaborative filtering, 2024. URL <https://arxiv.org/abs/2402.08426>.
- Bowen Zheng, Yupeng Hou, Hongyu Lu, Yu Chen, Wayne Xin Zhao, Ming Chen, and Ji-Rong Wen. Adapting large language models by integrating collaborative semantics for recommendation, 2023. URL <https://arxiv.org/abs/2311.09049>.
- Jieming Zhu, Mengqun Jin, Qijiong Liu, Zexuan Qiu, Zhenhua Dong, and Xiu Li. Cost: Contrastive quantization based semantic tokenization for generative recommendation, 2024. URL <https://arxiv.org/abs/2404.14774>.
- Tianyu Zhu, Yansong Shi, Yuan Zhang, Yihong Wu, Fengran Mo, and Jian-Yun Nie. Collaboration and transition: Distilling item transitions into multi-query self-attention for sequential recommendation, 2023. URL <https://arxiv.org/abs/2311.01056>.

A APPENDIX

A.1 CURRENT IMPLEMENTATION DETAILS

The current MGR-SID v1 implementation lives in an isolated experimental branch of the repository and does not alter the default MiniOneRec tokenizer path. The main training and generation entry points are:

- `scripts/experiment_mgr_sid_v1_train.py`
- `scripts/experiment_mgr_sid_v1_generate.py`

The core implementation is in:

- `src/onerec/experiments/mgr_sid/train_v1.py`
- `src/onerec/experiments/mgr_sid/transplanted_graph_bank.py`
- `src/onerec/experiments/mgr_sid/paper_transplants.py`

Table 9: Concrete graph-bank settings used by the current v1 experiments.

Component	Setting
History window	10
Coarse minimum edge weight	2.0
Local minimum edge weight	1.0
Semantic anchor top- k	32
Semantic anchor mixing weight	0.35
Spectral rank	48
Middle-band eigen ratio	[0.25, 0.65]
Temporal mixing ratio (alt. middle view)	0.35
Graph top- k per row in training	32

A.2 GRAPH CONSTRUCTION CHOICES IN V1

A.3 TOKENIZER TRAINING HYPERPARAMETERS

Table 10: Upstream-aligned tokenizer training regime used for the fair comparison in Table 3.

Hyperparameter	Value
Number of quantizers	3
Codebook sizes	[256, 256, 256]
Quantized embedding dimension	32
Encoder/decoder widths	[2048, 1024, 512, 256, 128, 64]
Optimizer	AdamW
Learning rate	10^{-3}
Weight decay	0
Scheduler	constant with warmup
Warmup epochs	50
Epochs	10000
Batch size	20480
Quantization loss weight	1.0
β	0.25
Graph loss weights	(0.05, 0.15, 0.05)

A.4 ADDITIONAL INTERPRETATION OF THE LOCAL AMBIGUITY RESULT

The local ambiguity analysis compares the final baseline index with the final hierarchy-aware index rather than comparing recommendation outputs. This makes the analysis tokenizer-native. The result is therefore best interpreted as evidence about the structure of the tokenizer itself:

- the hierarchy-aware tokenizer produces less crowded same- l_2 buckets;
- the reduction is visible both at catalog level and when weighted by test targets;
- the structural change is aligned with the motivating failure mode of local leaf ambiguity.

A.5 PRELIMINARY DOWNSTREAM SFT RESULT

The first downstream SFT study should be interpreted as *partial transfer evidence*, not as a finished end-to-end claim.

- The hierarchy-aware tokenizer improves short-range metrics on Industrial: NDCG@3 rises from 0.0777 to 0.0802, and HR@3 rises from 0.0882 to 0.0929.
- The same tokenizer remains weaker than the semantic baseline at @10+.
- Relative to historical SFT runs in the project CSV, both current runs still trail the strongest historical Industrial configuration.

A.6 EVALUATE-TIME ERROR-DISTRIBUTION SUMMARY

The downstream error analysis shows why the SFT result is mixed.

- same- l_1 and same- l_2 top-1 error rates both decrease under the hierarchy-aware tokenizer;
- the gain is strongest on crowded targets with target l_2 fanout at least 4;
- baseline same- l_2 errors are the most recoverable bucket under the new tokenizer;
- however, some examples that the baseline already routed into the correct local subtree are broken by the hierarchy-aware SID, which hurts broader @10+ ranking.

A.7 SCOPE OF THE CURRENT DRAFT

This paper should be read as a tokenizer-stage draft rather than a finished end-to-end paper. The remaining milestones are:

1. add repeated-seed verification for the final tokenizer comparison;
2. evaluate the hierarchy-aware tokenizer on Office under the same upstream-aligned regime;
3. improve the tokenizer-to-SFT transfer so that local ambiguity gains do not break already-correct local routing;
4. feed the improved SID into RL after the SFT interface is stabilized;
5. compare against more published tokenizer baselines once stronger downstream metrics are available.